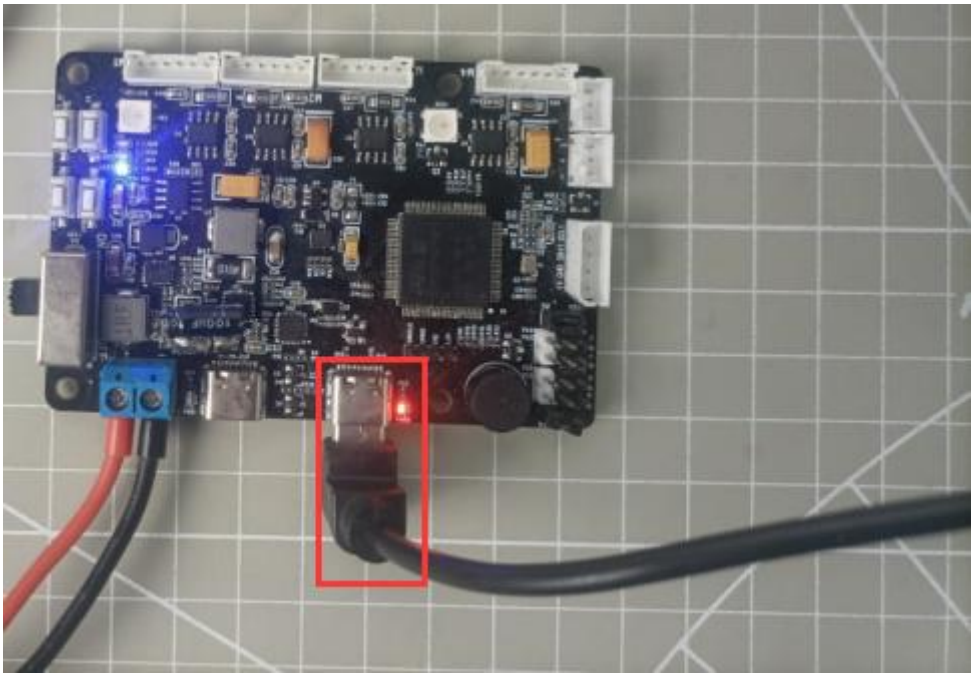
RRCLite Communication with the Host Computer - Sending Processing Source Code Analysis

1. Program Logic

This section mainly analyzes the source code for sending communication between the RRCLite development board and the host computer.

2. Hardware Connection

The hardware connection diagram is shown below:



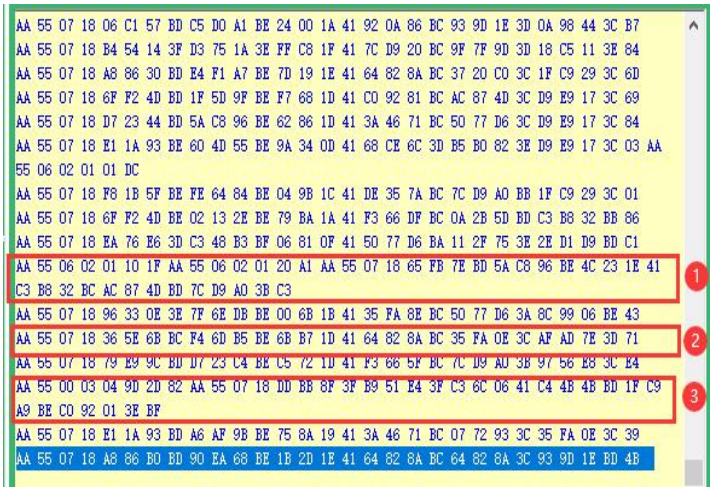
3. Program Download

The example for this section is “RosRobotControllerLite_ros”.

4. Program Outcome

After porting the ROS package, you can call the host computer interface to

receive messages from the car chassis bus servo, button messages, IMU messages, gamepad messages, and SBUS model remote control messages. After porting the ROS package, you can use a serial assistant to receive messages from the bus servo, button, and IMU on the car chassis. Please set the receiving code to HEX and turn on auto linefeed.



As shown in the figure above, when button K1 or K2 on the development board is pressed, the current event will be reported via the serial port, as shown in ①. The RRCLite development board will obtain and report IMU data in real-time, as shown in ②. The RRCLite tick timer will simultaneously report messages at fixed intervals, as shown in ③.

For a detailed analysis of the specific data frames, please refer to the previous lesson.

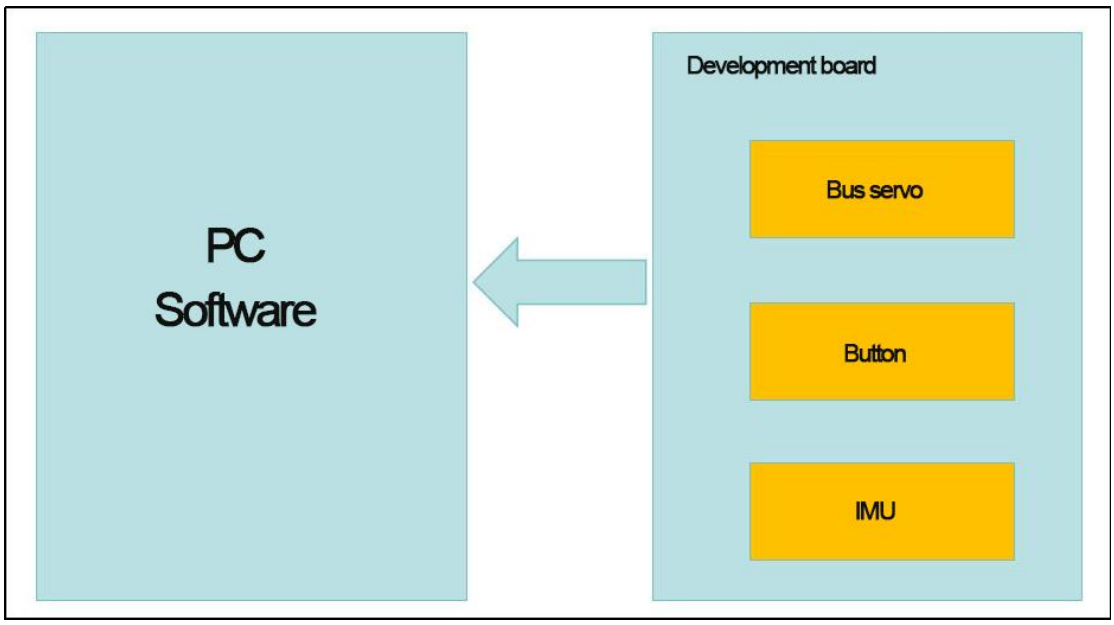
5. Pin Configuration

The development board's microcontroller uses UART3 serial port, converted to USB signal to communicate with the host computer through a conversion circuit. Pin information can be viewed in the stm32cubemx software. The corresponding chip pins are: PA9 and PA10.

6. Source Code Analysis

The program source code for this course is "**RosRobotControllerLite_ros**" .

The program framework is designed as shown below.



The messages sent by the development board to the host computer are divided into three categories: bus servo messages, button messages, and IMU messages.

6.1 Button Message Upload

Each button action corresponds to a button event. When a button event is triggered, the corresponding button event message is uploaded to the host computer.

In Hiwonder/System/app.c, the button_event_callback() function handles the event callback for buttons 1 and 2.

When a button action changes, it enters this callback function, stores the button ID and its triggered event in the report structure, and uploads the button message to the host computer via the packet_transmit() function.

```
void button_event_callback(ButtonObjectTypeDef *button,  ButtonEventIDEnum event)
{
    PacketReportKeyEventTypeDef report = {
        .key_id = button->id,
        .event = (uint8_t)(int)event,
    };
//  printf("Button:%d , event:%d\n",report.key_id, report.event);
    packet_transmit(    &packet_controller,
                        PACKET_FUNC_KEY,
                        &report,
                        sizeof(PacketReportKeyEventTypeDef));
    if(event == BUTTON_EVENT_CLICK) {
        buzzer_didi(buzzers[0], 2000, 50, 50, 1);
    }
}
```

By flashing the program, you can see the button ID and event type values sent to the host computer in the serial port 1 (displayed in hexadecimal (HEX) encoding format).

6.2 IMU Message Upload

After the IMU issues an interrupt, the microcontroller reads its current value and uploads it to the host computer.

In the Hiwonder/Porting/imu_porting.c file, the imu_task_entry() function is responsible for reading IMU data.

When this task reaches the osSemaphoreAcquire() function, it waits for a semaphore signal from the interrupt. Once the IMU issues an interrupt, the microcontroller interrupt sends a semaphore signal. When osSemaphoreAcquire() receives the signal, it continues executing the program, reads the IMU data, and uploads the message to the host computer via the packet_transmit() function.

In the Application/User/Core/stm32f4xx_it.c file, the IMU interrupt triggers the following microcontroller interrupt to send a semaphore signal.

```
/**
  * @brief This function handles EXTI line[15:10] interrupts.
  */
void EXTI15_10_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI15_10_IRQn 0 */
    extern osSemaphoreId_t IMU_data_readyHandle;
    if(__HAL_GPIO_EXTI_GET_IT(IMU_ITR_Pin) != RESET) {
        __HAL_GPIO_EXTI_CLEAR_IT(IMU_ITR_Pin);
        osSemaphoreRelease(IMU_data_readyHandle);
    }
  /* USER CODE END EXTI15_10_IRQn 0 */
  /* USER CODE BEGIN EXTI15_10_IRQn 1 */

  /* USER CODE END EXTI15_10_IRQn 1 */
}
```

The reading task is as below:

```
void imu_task_entry(void *argument)
{
    extern osSemaphoreId_t IMU_data_readyHandle;
  extern osMutexId_t oled_mutexHandle;

    if(begin() == 0)
    {
        //printf("qmi8658_init fail");
    }
    osDelay(100);
    PacketReportIMU_Raw_TypeDef report;
    for(;;) {
        //printf("test\n");
        osSemaphoreAcquire(IMU_data_readyHandle, osWaitForever);
        read_xyz(report.array.accel_array,report.array.gyro_array);
        /*printf("%d,%d,%d\r\n",(int)report.array.accel_array[0],
                               (int)report.array.accel_array[1],
                               (int)report.array.accel_array[2]);
     */
        packet_transmit(&packet_controller, PACKET_FUNC_IMU,
                        &report,
                        sizeof(PacketReportIMU_Raw_TypeDef));
        osDelay(50);
    }
}
```

By flashing the program, you can see the IMU attitude values sent to the host computer in the serial port 1 (displayed in hexadecimal (HEX) encoding format).

Note: The IMU sends data at a high frequency. It is best to turn off IMU printing when testing other messages to avoid interference.

6.3 Upload packet_transmit() Function Source Code Analysis

In the Hiwonder/Misc/packet.c file, the packet_transmit() function is defined as

follows:

```
int packet_transmit(struct PacketController *self, uint8_t func, void* data, size_t data_len)
{
    struct PacketRawFrame *p = packet_serialize(func, data, data_len);
    if(NULL != p) {
        return self->send_packet(self, p);
    }
    return -2;
}
```

- Parameter 1: Protocol controller object
- Parameter 2: Function code
- Parameter 3: Data to be sent
- Parameter 4: Length of the data

When sending data, you only need to pass in the default protocol controller object, defined in the Hiwonder/Porting/packet_porting.c file.

```
struct PacketController packet_controller;
```

The function codes are defined in the packet.h file, with nine available function codes.

```
enum PACKET_FUNCTION {
    PACKET_FUNC_SYS = 0,
    PACKET_FUNC_LED,
    PACKET_FUNC_BUZZER,
    PACKET_FUNC_MOTOR,
    PACKET_FUNC_PWM_SERVO,
    PACKET_FUNC_BUS_SERVO,
    PACKET_FUNC_KEY,
    PACKET_FUNC_IMU,
    PACKET_FUNC_GAMEPAD,
    PACKET_FUNC_SBUS,
    PACKET_FUNC_NONE,
};
```

Different messages have different input data. For detailed information, please refer to "Lesson 13 RRCLite Communication Protocol with the Host Computer Analysis".

1) The packet_serialize() function combines the function code, data, and data length into a protocol structure and returns it. The packet_serialize() function process is as follows:

- Create a structure variable and assign it to pointer p.
- Sequentially assign the protocol header, data length, function code, data, and CRC checksum to p.
- Return the structure pointed to by p.

```
struct PacketRawFrame* packet_serialize(uint8_t func, void* data_raw, size_t data_len)
{
    struct PacketRawFrame *p =  LWMEM_RAM_MALLOC(data_len + 6);
    if(NULL != p) {
        p->start_byte1 = PROTO_CONST_STARTBYTE1;
        p->start_byte2 = PROTO_CONST_STARTBYTE2;
        p->data_length = data_len;
        p->function = func;
        memcpy(p->data_and_checksum, data_raw, data_len);
        uint8_t checksum = checksum_crc8(&p->function, data_len + 2);
        p->data_and_checksum[data_len] = checksum;
    }
    return p;
}
```

2) The send_packet() structure member function uploads the assembled protocol data. The send_packet() function sends the data to the packet_tx queue.

```
static int send_packet(struct PacketController *self, struct PacketRawFrame *frame)
{
    return osMessageQueuePut(packet_tx_queueHandle, &frame, 0, 10);
}
```

The packet_tx_task_entry() protocol packet sending task function, defined in packet_porting.c, completes the data transmission to the host computer.

```
void packet_tx_task_entry(void *argument)
{
    for(;;) {
        osSemaphoreAcquire(packet_tx_idleHandle, osWaitForever); /* Waiting to send idle signal */
        osStatus_t status = osMessageQueueGet(packet_tx_queueHandle, &packet_controller.tx_dma_buffer, NULL, osWaitForever); /* Retrieve data from the sending queue */
        if(osOK == status) {
            HAL_UART_RegisterCallback(&huart1, HAL_UART_TX_COMPLETE_CB_ID, packet_dma_transmit_finished); /* Registration DMA reception completion callback */
            HAL_UART_Transmit_DMA(&huart1, (uint8_t*)packet_controller.tx_dma_buffer, packet_controller.tx_dma_buffer->data_length + 5);  /* Trigger DMA transmission*/
        }
    }
}
```

The protocol data is sent via the UART serial port to the conversion circuit, which converts the data to USB format and sends it to the host computer. This completes the entire protocol sending process.